VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Darshan Y.G. (1BM23CS087)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Darshan Y.G. (1BM23CS087),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Lab faculty Incharge Name Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	04-02-2026	Write a python program to import and export data using Pandas library functions	4
2	04-02-2026	Demonstrate various data pre-processing techniques for a given dataset	5-7
3	11-02-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	8-10
4	25-03-2026	Build Logistic Regression Model for a given dataset	16-17
5	11-03-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	13-15
6	25-02-2026	Build KNN Classification model for a given dataset.	11-12
7	08-4-2026	Build Support vector machine model for a given dataset	18-20
8	08-04-2026	Implement Random forest ensemble method on a given dataset.	21-22
9	01-04-2026	Implement Boosting ensemble method on a given dataset.	23-25
10	01-04-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	25-26
11	108-04-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	27-28

Github Link:

https://github.com/DarshanYG/Machine-Learning-Lab-5th-Sem

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot

Code:

```
# Import required libraries

import pandas as pd

from google.colab import files


data = {

 "Name": ["Ram", "Sam", "Riya"],

 "Age": [20, 21, 19],

 "Marks": [85, 90, 88]

}


# Create DataFrame

df_create = pd.DataFrame(data)


# Save as CSV

df_create.to_csv("data.csv", index=False)


print("Sample CSV file 'data.csv' created successfully!\n")

print("You can upload your own file OR use the created 'data.csv'")

uploaded = files.upload()
```

```
# If no file uploaded, use created file
if len(uploaded) == 0:
 file_name = "data.csv"
 print("Using default created file: data.csv")
else:
 file_name = list(uploaded.keys())[0]


if file_name.endswith('.csv'):
 df = pd.read_csv(file_name)
elif file_name.endswith('.xlsx'):
 df = pd.read_excel(file_name)
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot:

4/2/2026

Date __/__/__
Page _____

LAB-1

(1) Load the Data Set (Ans) No. of columns = 5

code:

```
from sklearn.datasets import load_iris
import pandas as pd
data = load_iris()
df = pd.DataFrame(data.data, column = data.feature_names)
df["species"] = data.target
df.head()
```

(2) Understand the slope, shape

code: (Ans) No. of rows = 150

```
df.shape
```

No. of columns = 5

output: (150, 5) → coloumns
→ no. of rows

(3) Explore columns, which column is the output?

code:

```
df.columns
df.info()
```

Ans) The column "species" is the output column.

(4) Which feature has the largest value?

→ df.describe()

Ans) The feature 'sepal length (cm)' has the largest value

(5) Are classes balanced?

→ df.['species'].value_counts()

Ans) Yes, the classes are balanced, with each class having 50 samples.

(6) Plot the graph. Do you see group forming?

Date __/__/__
Page _____

11/02/20

→ plt . scatter (df ['sepal length (cm)'] , df ['petal length (cm)'])
plt . xlabel ('sepal length')
plt . ylabel ('petal length')
plt . show ()

Ans) Yes, we can clearly see the group forming.

8) Sort the data. what happens to species label?
→ df.sort values (by = petal length (cm) . head()

Ans) When we sort the data by petal length, we can see the data samples with species = 0 at the top, Thus we can infer that the data samples with species = 0 have the smallest petal length

9) Compare two species. Do feature values differ?
→ df [df ["species"] == 0] . head ()
df [df ['species'] == 1] . head ()

Ans) Yes, for species 0, the values are slightly less or lower whereas for series 1, the input values are on the higher side
eg:- for species 0, the petal length is around 1.5cm where as it is 4.5 cm for species 1.
Only the sepal width is ~~about~~ almost similar for all different species.

Code:

```
from sklearn.datasets import load_iris

import pandas as pd

import matplotlib.pyplot as plt

# Load data

data = load_iris()

df = pd.DataFrame(data.data, columns=data.feature_names)

df["species"] = data.target

# Basic info

print(df.head())

print(df.shape)

print(df.info())

# Describe

print(df.describe())

# Class balance

print(df["species"].value_counts())

# Scatter plot

plt.scatter(df["sepal length (cm)"], df["petal length (cm)"])

plt.xlabel("Sepal Length")

plt.ylabel("Petal Length")

plt.show()

# Sorting

print(df.sort_values(by="petal length (cm)").head())
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset.

Screenshot:

11/02/2026

LAB-2

End to End Machine Learning lab

House price prediction

Step-① Supervised Learning Regression → Learn from labelled data and we are predicting a continuous numerical value

→ we need to build a ML model to predict median house value in California.

→ RMSE: Measures average magnitude of prediction errors

→ MAE: Measures the average absolute differen between actual and predicted values.

→ R^2 Score: Gives the proportion of variance in the dependant variable (0 to 1), closer to 1 = fit

Step2: Get Data

size ⇒ (20640, 9)

Step3: Discover data.

→ There are no missing values in the dataset

→ df.describe() gives the mean, min, max, count, etc

Step4: Visualize the data

→ Distribution of houseprice

Most houses are concentrated on "2" price range

→ Income vs house price. as income Increases, house price Increases.

→ correlation heatmap
MedInc has strongest positive correlation with med house val.
→ Location visualization.

Step 5: Preparing the data.
Train test split.
80% of data for training.
20% of data for testing.
feature scaling → required for linear regression
Converts data to mean = 0
std = 1
Because some features have large values (pop.) and some small.

Step (6): Select & train Base model (Linear Regression)
RMSE = 0.7449
MAE = 0.53630
R^2 = 0.5765

Step (7): Train advanced models.
# Decision tree Regressor.
RMSE = 0.7138
Since rmse is lower, better performance than linear regression.
# Random forest regression.
rmse = 0.5129.
better performance.

Step 8: fine-Tune the model
Hyper parameter ~~fin~~ tuning.
rmse = 0.5132.

Step 9: Model comparison.

Model	RMSE
Linear regression	0.744934
Decision Tree	0.713838
Random forest	0.512864
Tuned Random forest	0.513263

Random forest → Lowest RMSE

Code:

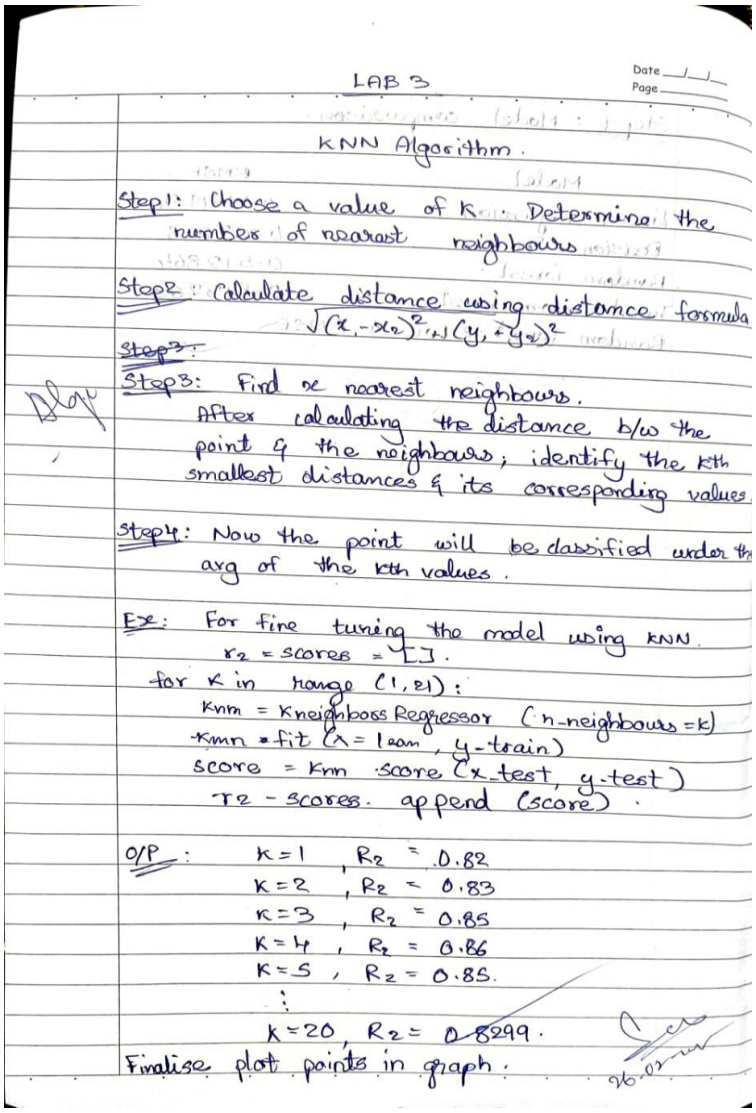
```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
```

```
from sklearn.tree import DecisionTreeRegressor

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error

data = fetch_california_housing()

X = data.data

y = data.target

#Split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# Models

lr = LinearRegression().fit(X_train, y_train)

dt = DecisionTreeRegressor().fit(X_train, y_train)

rf = RandomForestRegressor().fit(X_train, y_train)


# Evaluation

for name, model in [("LR", lr), ("DT", dt), ("RF", rf)]:

 pred = model.predict(X_test)

 rmse = np.sqrt(mean_squared_error(y_test, pred))

 print(name, "RMSE:", rmse)
```

Program 6:

Build KNN Classification Model for a given dataset

Screenshot:



LAB 3

KNN Algorithm.

Step1: Choose a value of K. Determine the number of nearest neighbours

Step2: Calculate distance using distance formula
$\sqrt{(x_1-x_2)^2+(y_1-y_2)^2}$

~~Step3:~~

Step3: Find K nearest neighbours.
After calculating the distance b/w the point & the neighbours; identify the Kth smallest distances & its corresponding values.

Step4: Now the point will be classified under the avg of the Kth values.

Ex: For fine tuning the model using KNN.

```
r2 = scores = []
for K in range (1,21):
    Knn = KNeighborsRegressor (n_neighbours=k)
    Knn.fit (x_train, y_train)
    score = Knn.score (x_test, y_test)
    r2_scores.append (score)
```

O/P:
K=1, R_2 = 0.82
K=2, R_2 = 0.83
K=3, R_2 = 0.85
K=4, R_2 = 0.86
K=5, R_2 = 0.85.
⋮
K=20, R_2 = 0.8299.

Finalise plot points in graph.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsRegressor

from sklearn.metrics import r2_score
```

```
data = fetch_california_housing()

X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

scores = []

k_values = range(1, 21)

for k in k_values:

 model = KNeighborsRegressor(n_neighbors=k)

 model.fit(X_train, y_train)

 pred = model.predict(X_test)

 score = r2_score(y_test, pred)

 scores.append(score)

 print(f"K={k}, R2={score:.4f}")


plt.plot(k_values, scores)

plt.xlabel("K")

plt.ylabel("R2 Score")

plt.show()
```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot

LAB 4

Date __/__/__
Page _____

* Implement Decision Tree classification (ID3) and regression (CART) and test with more than one dataset and measure the metrics.

Decision Tree (ID3):

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer,
load_wine
from sklearn.tree import DecisionTreeClassifier.

data = load_breast_cancer()
x = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_
split( x, y, test_size = 0.3, random_state = 42)

model = DecisionTreeClassifier (criterion = 'entropy')
model.fit (x_train, y_train)
y_pred = model.predict (x_test)

print ("Accuracy", accuracy_score (y_test, y_pred))
print (confusion_matrix (y_test, y_pred))
print (classification_report (y_test, y_pred))
```

O/P:

```
Accuracy : 0.9532163742
[[57  6]
 [2 106]]
```

Classification report:

	precision	recall	f1score	support
0	0.97	0.90	0.93	63
1	0.95	0.98	0.96	108
accuracy			0.95	171
macro avg.	0.96	0.94	0.95	171
weighted avg	0.95.	0.95	0.95	171

2nd Dataset.

Accuracy : 0.88888888

```
[[19  0  0]
 [2  18  1]
 [2  1  11]]
```

Classification Report:

	precision	recall	f1score	support
0	0.83	1.00	0.90	19
1	0.95	0.86	0.90	21
2	0.92	0.79	0.85	14
accuracy			0.89	54
macro avg.	0.90	0.88	0.88	54
weighted avg	0.90	0.89	0.89	54.

REGRESSION (CART):

```
data = fetch_california_housing()
x = data.data
y = data.target

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
reg = DecisionTreeRegressor()
reg.fit(x_train, y_train)
y_pred = reg.predict(x_test)

print("Mean square Error", mean_squared_error(y_test, y_pred))
print("R2 score:", r2_score(y_test, y_pred))
```

O/P:

Mean_squared_error: 0.532422564447

R2 score : 0.594357108857

Diabetes dataset

Mean_squared_error : 5335.278195

R2 score : 0.011675880085

Code:

```
from sklearn.datasets import load_breast_cancer, load_wine, fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

from sklearn.metrics import accuracy_score, confusion_matrix, mean_squared_error

# Classification

for dataset in [load_breast_cancer(), load_wine()]:

 X, y = dataset.data, dataset.target
```

```
 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = DecisionTreeClassifier(criterion="entropy")
 model.fit(X_train, y_train)

pred = model.predict(X_test)
 print("Accuracy:", accuracy_score(y_test, pred))
 print("Confusion Matrix:\n", confusion_matrix(y_test, pred))

# Regression
data = fetch_california_housing()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = DecisionTreeRegressor()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, pred))
```

Program 4

Build Logistic Regression Model for a given dataset

Screenshot:

5/03/26

LAB - 5

Date __/__/__
Page

LOGISTIC REGRESSION

HR_Comma_sep.csv

Bar graph:

model accuracy : 0.766

confusion matrix :

$$\begin{bmatrix} 2118 & 176 \\ 533 & 173 \end{bmatrix}$$

Classification report:

	Precision	recall	f1-score	support
0	0.80	0.92	0.86	2294
1	0.50	0.25	0.33	706

Date __/__/__
Page

1. Which variables did you identify as having a direct and clear impact on employee retention?
   - Satisfaction Level
   - satisfaction level ∝ 1 / retention
   - Average_monthly_hours
   - Time_spent
   - Work_accident
   - Salary

2. What was the accuracy of the logistic regression model? Do you think it is good? Why?

   Accuracy of the model is 76.66%. It is good because dataset is real world and not ~~idet~~ ideal.

   Logistic regression is a simple model.

(2) Zoo - dataset:

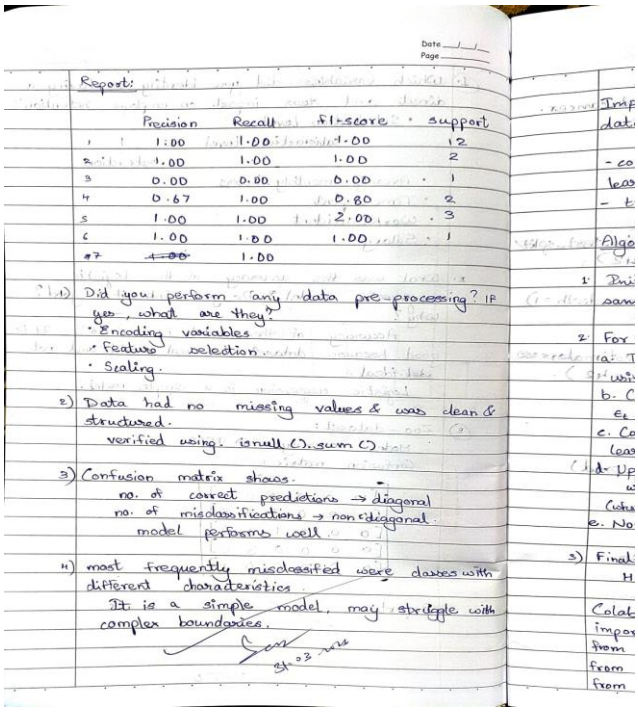
Model accuracy : 0.9523

Confusion matrix :

$$\begin{bmatrix} 12 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Report:

P.T.O.



Report:

	Precision	Recall	f1-score	support
1	1.00	1.00	1.00	12
2	1.00	1.00	1.00	2
3	0.00	0.00	0.00	1
4	0.67	1.00	0.80	2
5	1.00	1.00	2.00	3
6	1.00	1.00	1.00	1
~~#7~~	~~1.00~~	1.00		

1.1) Did you perform any data pre-processing? If yes, what are they?
- Encoding variables
- Feature selection
- Scaling.

2) Data had no missing values & was clean & structured.
verified using: isnull().sum()

3) Confusion matrix shows.
no. of correct predictions → diagonal
no. of misclassifications → non-diagonal
model performs well

4) most frequently misclassified were classes with different characteristics
It is a simple model, may struggle with complex boundaries.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("HR_comma_sep.csv")

df = pd.get_dummies(df, columns=["salary"], drop_first=True)

X = df.drop("left", axis=1)
y = df["left"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

Problem 7

Build support vector machine model for a given dataset.

Screenshot

LAB-7

Date __/__/__
Page _____

Building a support vector machine model for a given dataset

Algorithm:-

1) Input Dataset
   - Load dataset with features (X) and lables (y)
2) Preprocess Data.
   - Handle missing values
   - Split into training and testing sets
   - Normalize / standardize features.
3) Choose SVM model
   - Select Kernel
4) Train Model
   - Fit SVM model on training data.
5) Make Predictions
   - Use trained model to predict test data
6) Evaluate Model
   - Calculate accuracy / confusion matrix

Colab code implementation:

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.SVM import SVC
from sklearn.metrics import accuracy_score, confusion_matr
data = datasets.load_iris()
X = data.data
Y = data.target
```

Date __/__/__
Page _____

```
X_train , X_test , Y_train , Y_test =
= train_test_split (X, Y, testsize=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform (X_train)
X_test = scaler.transform (X_test)

model = SVC (kernel='linear')

model.fit (X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix (y_test, y_pred)

print("Accuracy:", accuracy)
print("Confusion Matrix:\n", cm)
```

O/P:

```
Predicted values : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1
 2 2 1 2 2 2 2 2 2 1]

Actual values: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 2
 1 2 2 2 2 2 2 2 1]

Accuracy : 1.0
Confusion Matrix: [[10  0  0]
                   [ 0  9  0]
                   [ 0  0 11]]
```

Code:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

data = datasets.load_iris()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SVC(kernel='linear')
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

Problem 8

Implement Random Forest ensemble method on a given dataset

Screenshot

LAB 7

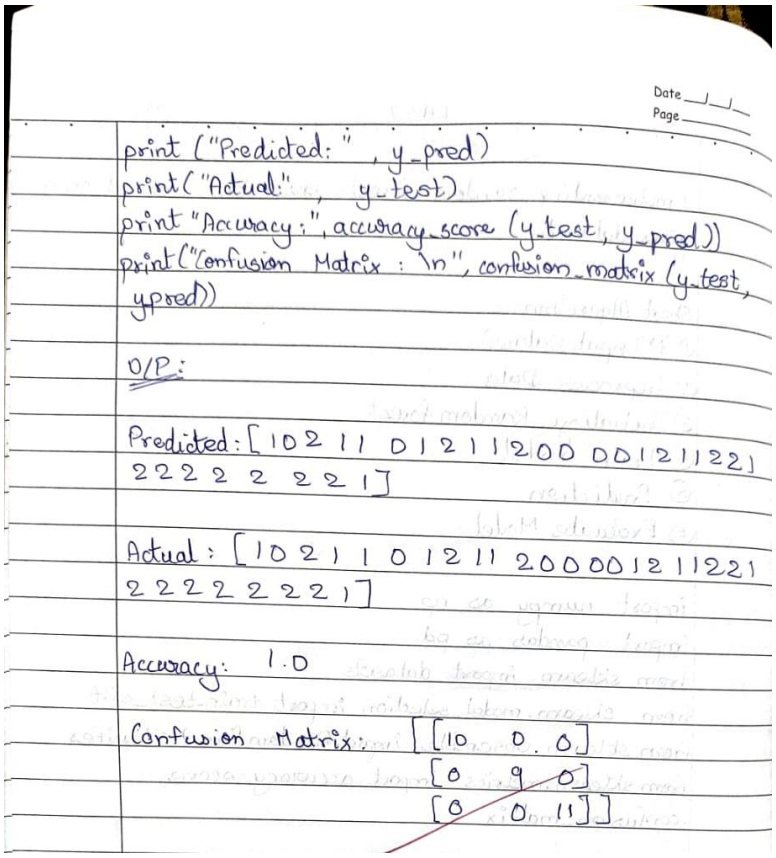
Date __/__/__
Page _____

Implementing random forest ensemble method on a given dataset.

Short Algorithm:

1. Input Dataset
2. Preprocess Data
3. Initialize Random forest
4. Train model
5. Prediction
6. Evaluate Model.

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,
confusion_matrix

data = datasets.load_iris()
X = data.data
y = data.target

X_train, X_test, Y_train, Y_test = train_test_split
(X, Y, test_size = 0.2, random_state = 42)

model = RandomForestClassifier(n_estimators = 100,
random_state = 42)

model.fit (X_train, y_train)

Ypred = model.predict (X_test)
```



```
print ("Predicted: " , y_pred)
print("Actual:" , y_test)
print "Accuracy:", accuracy_score (y_test, y_pred))
print("Confusion Matrix : \n", confusion_matrix (y_test,
ypred))
```

O/P:

```
Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 2 1
2 2 2 2 2 2 2 1]

Actual: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 2 1
2 2 2 2 2 2 2 1]

Accuracy: 1.0

Confusion Matrix: [[10 0 0]
                   [0 9 0]
                   [0 0 11]]
```

Code:

```
from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score

data = datasets.load_iris()

X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier(n_estimators=100)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
```

Problem 9

Implement Boosting ensemble method on a given dataset.

Screenshot

Date __/__/__
Page _____

LAB - 6 (A)

Implement Boosting ensemble method on a given dataset.

- combines several weak learners into a strong learner.
- trains models sequentially

Algorithm:

1. Initialize equal weights for all N training samples: $w_i = \frac{1}{N}$ for $i = 1$ to N

2. For $t = 1$ to T (number of weak learners):
   a. Train a weak learner h_t (e.g., Decision Stump) using current weights.
   b. Compute the weighted error of h_t:
   $\epsilon_t = \sum_{i=1}^{N} w_i \cdot \mathbb{I}(y_i \neq h_t(x_i))$
   c. Compute the model weight (importance) of this learners: $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$
   d. Update sample weights
   $w_i \leftarrow w_i \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))$
   (where $y_i, h_t(x_i) \in \{-1, +1\}$)
   e. Normalize the weights so they sum to 1.

3) Final strong classifier:
   $H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \cdot h_t(x)\right)$.

Colab Implementation:

```
import numpy as np
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_breast_cancer.

data = load_breast_cancer()
X = data.data
y = data.target

y = np.where(y == 0, -1, 1)
X_train, X_test, y_train, y_test = train_test_split
    X, y, test_size = 0.2, random_state = 42)

base_estimator = DecisionTreeClassifier(max_depth = 1)

model = AdaBoostClassifier(
        estimator = base_estimator, n_estimators =
        learning_rate = 1.0, random_state = 42 ).

model.fit (X_train, y_train)

y_pred = model.predict (X_test)
accuracy = accuracy_score (y_test, y_pred)
```

O/P:

```
Test Accuracy : 0.9737
Number of weak learners used: 50
```

Code:

```
import numpy as np

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score

data = load_breast_cancer()

X, y = data.data, data.target

y = np.where(y == 0, -1, 1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

base = DecisionTreeClassifier(max_depth=1)

model = AdaBoostClassifier(estimator=base, n_estimators=50)
```

model.fit(X_train, y_train)

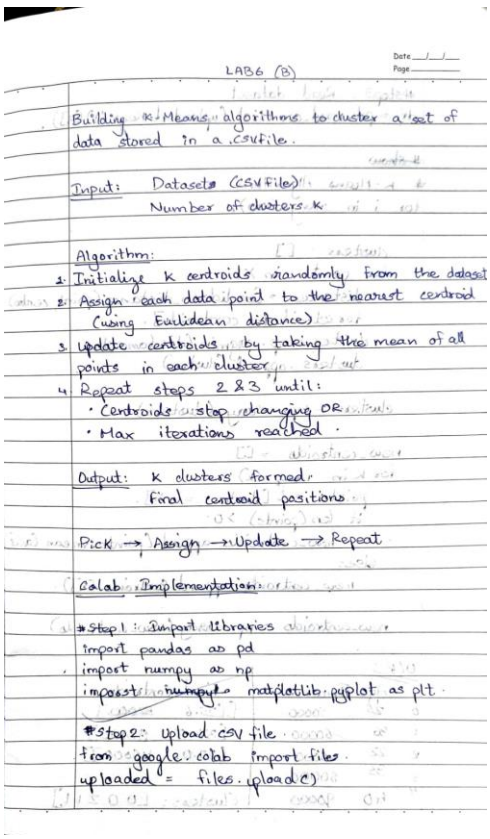
pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))

Problem 10

Building K-Means algorithm to cluster a set of data stored in a csv file.

Screenshot



LAB6 (B)

Building K-Means algorithms to cluster a set of data stored in a .csv file.

Input: Dataset (CSV File)
Number of clusters K

Algorithm:
1. Initialize K centroids randomly from the dataset
2. Assign each data point to the nearest centroid (using Euclidean distance)
3. Update centroids by taking the mean of all points in each cluster
4. Repeat steps 2 & 3 until:
   - Centroids stop changing OR
   - Max iterations reached.

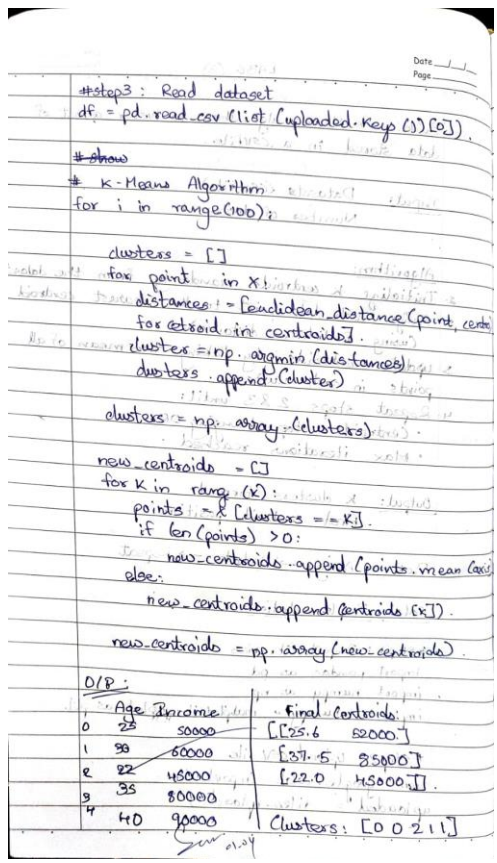
Output: K clusters formed
Final centroid positions

Pick → Assign → Update → Repeat

Colab Implementation:

#Step 1: Import libraries
import pandas as pd
import numpy as np
~~imposst numpy~~ matplotlib.pyplot as plt

#Step 2: Upload csv file
from google.colab import files
uploaded = files.upload()



#Step3: Read dataset
df = pd.read_csv(list(uploaded.keys()[0]).

~~#show~~
# K-Means Algorithm
for i in range(100):

clusters = []
for point in X:
distances = [euclidean_distance(point, centr
for centroid in centroids].
cluster = np.argmin(distances)
clusters.append(cluster)

clusters = np.array(clusters)

new_centroids = []
for k in range(k):
points = X[clusters == k].
if len(points) > 0:
new_centroids.append(points.mean(axis
else:
new_centroids.append(centroids[k]).

new_centroids = np.array(new_centroids).

O/P:

	Age	Income
0	23	50000
1	38	60000
2	22	45000
3	35	80000
4	40	90000

Final Centroids:
[[25.6 52000.]
[37.5 85000.]
[22.0 45000.]].

Clusters: [0 0 2 1 1]

Code:

import numpy as np

import pandas as pd

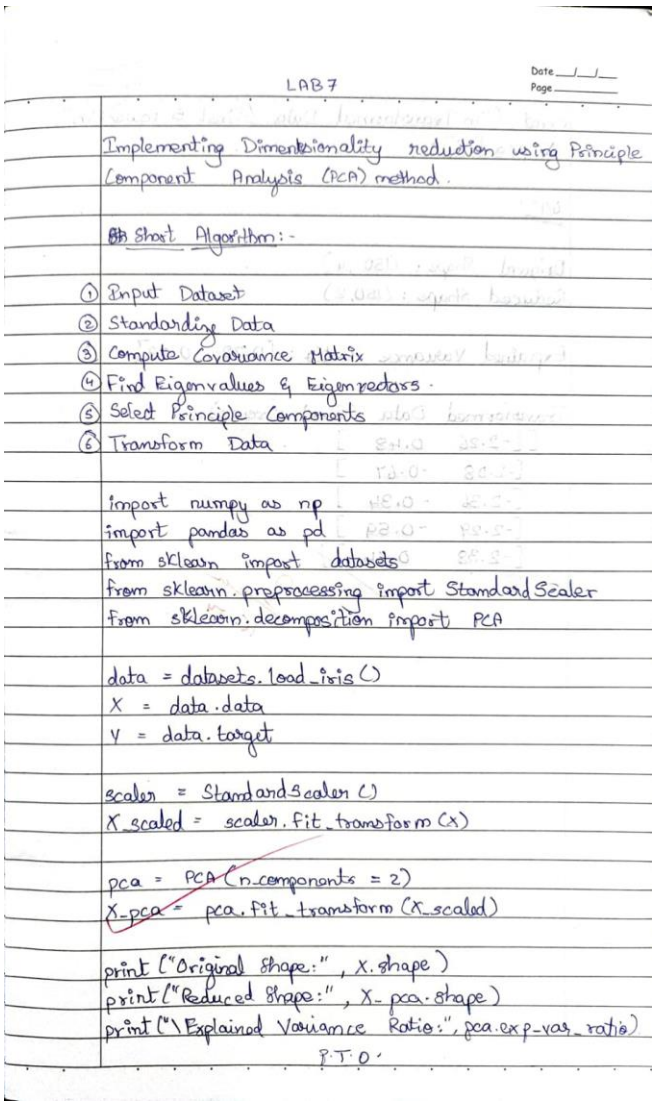
import matplotlib.pyplot as plt

```
from sklearn.cluster import KMeans

df = pd.read_csv("your_file.csv")

X = df.iloc[:, :2].values

kmeans = KMeans(n_clusters=3)

kmeans.fit(X)

print("Centroids:\n", kmeans.cluster_centers_)

print("Labels:", kmeans.labels_)

plt.scatter(X[:,0], X[:,1], c=kmeans.labels_)

plt.scatter(kmeans.cluster_centers_[:,0], kmeans.cluster_centers_[:,1], c='red')   plt.show()
```

Problem 11

Implementing Dimensionality reduction using Principle Component Analysis (PCA) method.

Screenshot



LAB 7

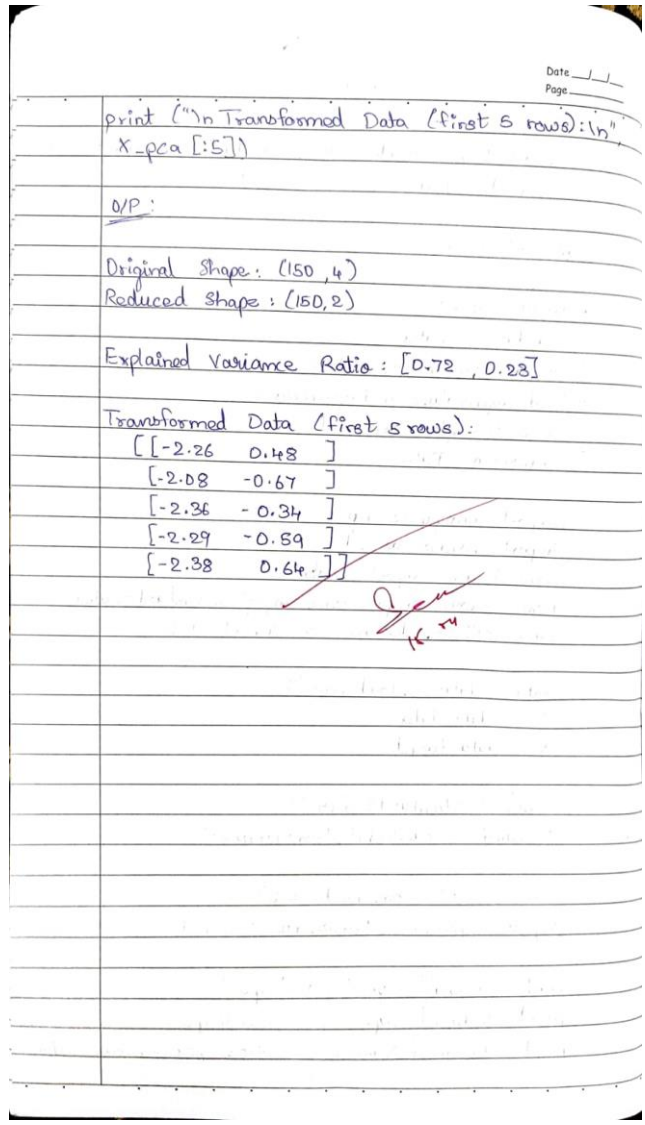
Implementing Dimensionality reduction using Principle Component Analysis (PCA) method.

Short Algorithm:-

1. Input Dataset
2. Standarding Data
3. Compute Covariance Matrix
4. Find Eigenvalues & Eigenvectors.
5. Select Principle Components
6. Transform Data

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data = datasets.load_iris()
X = data.data
y = data.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components = 2)
X_pca = pca.fit_transform(X_scaled)

print("Original Shape:", X.shape)
print("Reduced Shape:", X_pca.shape)
print("\Explained Variance Ratio:", pca.exp_var_ratio)
```

P.T.O.



```
print("\n Transformed Data (first 5 rows):\n",
X_pca[:5])
```

O/P:

```
Original Shape: (150, 4)
Reduced Shape: (150, 2)

Explained Variance Ratio: [0.72, 0.23]

Transformed Data (first 5 rows):
[[-2.26  0.48 ]
 [-2.08 -0.67 ]
 [-2.36 -0.34 ]
 [-2.29 -0.59 ]
 [-2.38  0.64 ]]
```

Code:

```
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data = datasets.load_iris()
X = data.data
```

```
X = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

print("Original Shape:", X.shape)
print("Reduced Shape:", X_pca.shape)
print("Explained Variance:", pca.explained_variance_ratio_)

print(X_pca[:5])
# Useful for viva: compare models quickly
from sklearn.metrics import accuracy_score
print("Use accuracy_score, confusion_matrix, r2_score depending on task")
```